



SLITHERLINK R

Rapport Technique Complet — Package R & Application Shiny

Auteurs

Dossou AGOSSOU
Moussa DIAGNE

Formation

M1 Statistique et Science des
Donnees
Universite de Montpellier | 17
Avril 2026

1. Introduction

Ce rapport presente le developpement du package R SlitherlinkR, realise dans le cadre de l'unite d'enseignement Programmation R du Master 1 Statistique et Science des Donnees de l'Universite de Montpellier.

L'objectif etait de concevoir un package R complet permettant de generer, jouer et resoudre le jeu logique Slitherlink, accessible via une application Shiny interactive professionnelle.

1.1 Le jeu Slitherlink

Le Slitherlink est un casse-tete logique japonais base sur les regles suivantes :

- unique et fermee (ni croisement, ni ramification)** La boucle doit etre
- 0 ou 2 segments** Chaque sommet a exactement
- combien de ses 4 cotes appartiennent a la boucle** Le chiffre dans une case indique
- n'ont aucune contrainte** Les cases sans chiffre (NA)

1.2 Objectifs du projet

- Developper une bibliotheque R avec une architecture de package standard
- Implementer un moteur de verification des regles du jeu
- Creer un solveur automatique (ILP et C++)

- 4. Generer des puzzles avec unicite de solution garantie
- 5. Deployer une application Shiny interactive avec leaderboard
- 6. Assurer la qualite via 43 tests unitaires et CI/CD

2. Architecture du Package

2.1 Structure des fichiers

Organisation du depot GitHub

R/grid_management.R Classe S3, constructeur, toggle des segments

R/grid_plot.R Visualisation ggplot2

R/validator.R Moteur de verification (DFS)

R/solver.R Solveur ILP (lpSolve)

R/solver_interface.R Interface solveur

R/generator.R Generateur de puzzles (unicite garantie)

R/database.R Leaderboard SQLite

R/run_app.R Lancement application Shiny

src/solver.cpp Solveur backtracking C++ (Rcpp)

shiny-app/app.R Application Shiny complete

tests/testthat/ 5 fichiers de tests (43 tests au total)

man/ 18 fichiers de documentation roxygen2

.github/workflows/ GitHub Actions CI/CD

2.2 Repartition du travail

Phase	Contenu	Auteur	Branche	Statut
Phase 1	Structure S3, ggplot2, toggle	Dossou AGOSSOU	main	Termine
Phase 2	Valideur, DFS, 26 tests	Moussa DIAGNE	dev2	Termine
Phase 3	Solveur C++ Rcpp	Dossou AGOSSOU	main	Termine
Phase 4	ILP, generateur, Shiny, SQLite	Moussa DIAGNE	dev2	Termine
Phase 5	Tests, doc, unicite, CI/CD	Dossou AGOSSOU	dev2	Termine

3. Modelisation Mathematique

3.1 Theorie des graphes

Le Slitherlink est modelise comme un graphe non oriente $G = (V, E)$ ou :

- : les $(n+1) \times (m+1)$ points d'intersection d'une grille $n \times m$ V (sommets)
- : tous les segments possibles entre deux points voisins E (aretes)
- : variable binaire — 1 si l'arete est tracee, 0 sinon $x[e] \in \{0, 1\}$

3.2 Classe S3 slitherlink

La structure de donnees centrale est un objet S3 contenant :

Representation en R

`n, m` : dimensions de la grille (nombre de lignes et colonnes de cases)
`indices` : matrice $n \times m$ des chiffres (0, 1, 2, 3 ou NA)
`h_edges` : matrice $(n+1) \times m$ des segments horizontaux
`v_edges` : matrice $n \times (m+1)$ des segments verticaux

Etats d'un segment :

- 0L -> absent (non trace)
- 1L -> trace (fait partie de la boucle)
- 2L -> barre (marque impossible par le joueur)

3.3 Contraintes mathematiques

Contrainte de degre des sommets

A chaque point d'intersection, le nombre de segments traces doit etre exactement 0 ou 2. Jamais 1 ni 3. C'est ce qui garantit l'absence de bouts libres dans la boucle.

Contrainte des cases (chiffres)

Si une case contient le chiffre k , exactement k de ses 4 cotes (haut, bas, gauche, droite) doivent etre traces. Les cases sans chiffre (NA) n'ont aucune contrainte.

Contrainte de boucle unique

Les deux contraintes precedentes ne suffisent pas a garantir une seule boucle. L'algorithme DFS parcourt le graphe des segments traces depuis un sommet actif et verifie que tous les autres sommets actifs sont atteints dans la meme composante connexe.

4. Phases de Developpement

4.1 Phase 1 — Fondations du Package (Dossou AGOSSOU)

La Phase 1 a pose les bases architecturales et visuelles du projet.

Classe S3 et constructeur

La fonction `new_slitherlink(indices)` cree l'objet central du jeu :

```
new_slitherlink <- function(indices) {  
  # Validation : matrice avec valeurs 0,1,2,3 ou NA  
  # Creation de h_edges (n+1 x m) et v_edges (n x m+1)  
  class(res) <- 'slitherlink'  
}
```

Toggle des segments

Les fonctions `toggle_h_edge()` et `toggle_v_edge()` implementent le cycle 0 -> 1 -> 2 -> 0 avec validation des coordonnees :

```
grid$h_edges[r, c] <- (grid$h_edges[r, c] + 1L) %% 3L
```

Visualisation ggplot2

La fonction `plot_slitherlink()` transforme la structure de donnees en image via un pipeline ggplot2 :

- `expand.grid()` pour les points d'intersection
- `as.data.frame(as.table())` pour les indices des cases
- `which(..., arr.ind=TRUE)` pour extraire les segments avec leur statut
- `geom_segment + geom_point + geom_text` pour le rendu final
- Couleurs : segment absent (`#1e2540`), trace (`#6366f1`), barre (`#ef4444`)

4.2 Phase 2 — Moteur de Verification (Moussa DIAGNE)

La Phase 2 a implemente toute la logique de validation du jeu avec 4 fonctions interconnectees.

`check_cells(grid)`

Parcourt toutes les cases avec un chiffre et verifie que la somme de ses 4 cotes traces correspond exactement au chiffre indique. Retourne `list(ok, errors)` avec le detail des cases incorrectes.

`check_degree(grid)`

Parcourt tous les $(n+1) \times (m+1)$ sommets et verifie que chaque sommet a exactement 0 ou 2 segments traces. Un sommet de degre 1 ou 3 indique une boucle mal formee.

`check_loop(grid)` — Algorithme DFS

Utilise un parcours en profondeur (Depth-First Search) pour verifier l'unicite de la boucle :

7. Construction de la liste d'adjacence des sommets connectes par un segment trace

8. DFS depuis le premier sommet actif
9. Verification que tous les sommets actifs sont dans la meme composante connexe
10. Si plusieurs composantes : plusieurs boucles distinctes detectees

validate_solution(grid)

Fonction principale qui appelle les 3 verifications et retourne un resultat global avec des messages explicites pour le joueur.

Tests unitaires — Phase 2

Fichier	Ce qui est teste	Nb tests
test-grid.R	Structure S3, toggle, coordonnees invalides	8
test-validator.R	check_cells, check_degree, check_loop, validate	18

4.3 Phase 3 — Solveur C++ Rcpp (Dossou AGOSSOU)

La Phase 3 a implemente un solveur haute performance en C++ compile via Rcpp.

Algorithme de backtracking

Le solveur C++ explore recursivement toutes les combinaisons possibles d'aretes avec un elagage (pruning) agressif :

- : **verifie les contraintes des cases partiellement assignees** `check_cell_partial()`
- : **verifie les degres des sommets en cours d'assignation** `check_vertex_partial()`
- : **verifie uniquement les cellules et sommets affectes par l'arete courante** `check_affected()`
- : **DFS final pour garantir l'unicite de la boucle** `check_single_loop()`

Integration Rcpp

La configuration Rcpp necessite :

- : **code C++ avec attributs** `// [[Rcpp::export]]` `src/solver.cpp`
- : **@useDynLib + @importFrom Rcpp sourceCpp** `R/SlitherlinkR-package.R`
- : **generes automatiquement par Rcpp::compileAttributes()** `src/RcppExports.cpp` et `R/RcppExports.R`

4.4 Phase 4 — Application Complète (Moussa DIAGNE)

Solveur ILP avec IpSolve

Le solveur principal utilise la Programmation Lineaire en Nombres Entiers via le package IpSolve :

Formulation ILP

Variables : $x[e]$ in $\{0,1\}$ pour chaque arete + $y[v]$ in $\{0,1\}$ pour chaque sommet

Contrainte cases : $\text{sum}(4 \text{ aretes de la case}) = \text{chiffre_k}$

Contrainte degre : $\text{sum}(\text{aretes de } v) = 2 * y[v]$

Elimination tours : $\text{sum}(\text{aretes du sous-tour}) \leq |\text{sous-tour}| - 1$

Resolution iterative : resoudre -> detecter sous-tours -> interdire -> repeter

Performances du solveur ILP

Taille grille	Solveur utilise	Temps moyen	Resultat
3 x 3	ILP IpSolve	~0.03 secondes	Valide
4 x 4	ILP IpSolve	~0.14 secondes	Valide
5 x 5	ILP IpSolve	~0.15 secondes	Valide

Generateur de puzzles

La fonction `generate_puzzle(h_sol, v_sol, difficulty)` genere des puzzles jouables :

11. Partir d'une boucle valide connue
12. Calculer tous les indices depuis la boucle
13. Masquer aleatoirement des cases selon la difficulte (30/50/70% de NA)
14. Verifier l'unicite de la solution via `is_unique_solution()`
15. Si non unique : recommencer jusqu'a `max_attempts` tentatives

Unicite garantie — `is_unique_solution()`

La fonction verifie mathematiquement qu'il n'existe pas de deuxieme solution :

16. Resoudre le puzzle une premiere fois (ILP)
17. Interdire cette solution : $\text{sum}(\text{aretes_sol1}) \leq |\text{sol1}| - 1$
18. Tenter de resoudre a nouveau
19. Si infaisable : solution unique confirme
20. Si une 2eme solution existe : recommencer la generation

Application Shiny

L'application offre une experience de jeu complete avec un design premium :

Fonctionnalites de l'application

Interface : theme sombre geometrique (Space Mono + DM Sans)

Niveaux : Facile 3x3 / Moyen 4x4 / Difficile 5x5

Puzzles : 3 formes par niveau (bord exterieur, L, peninsule) + unicite garantie

Timer : reactif mis a jour chaque seconde via `invalidateLater()`

Verifier : appelle `validate_solution()` avec feedback colore
Resoudre : appelle `solve_slitherlink()` ILP avec solution instantanee
Reinitialiser : efface la grille et redemarrage du timer
Leaderboard : SQLite persistant, top 8, sauvegarde et reset
Segments : 3 etats (trace violet / barre rouge / absent)

Base de donnees SQLite

Le leaderboard utilise RSQLite via DBI pour stocker les scores :

```
CREATE TABLE scores (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  pseudo TEXT, temps REAL, difficulte TEXT, date TEXT  
)
```

4.5 Phase 5 — Finalisation (Dossou AGOSSOU)

La Phase 5 a finalise la qualite du package avec tests complets, documentation et CI/CD.

Tests unitaires complets

Fichier	Contexte teste	Nb tests
test-grid.R	Structure S3, toggle, validation coordonnees	8
test-validator.R	check_cells, check_degree, check_loop, validate	18
test-generator.R	generate_puzzle, niveaux de difficulte, unicite	8
test-solver.R	solve_slitherlink ILP 3x3 et 4x4, puzzle impossible	5
test-database.R	init_db, save_score, colonnes table	4
TOTAL		43

GitHub Actions CI/CD

Un workflow GitHub Actions a ete configure pour lancer automatiquement R CMD check a chaque push sur main ou dev2 :

- Installation automatique de R 4.4.0 sur Windows
- Installation de toutes les dependances via `setup-r-dependencies`
- Execution de R CMD check avec `error-on: error`
- Badge dynamique sur le README indiquant le statut en temps reel

5. Resultats et Metriques Finales

Resultats devtools::check()

0 errors

(aucune erreur de code ou de structure)

0 warnings

(aucun avertissement)

0 notes

(aucune remarque)

43 tests

FAIL 0 | WARN 0 | SKIP 0 | PASS 43

GitHub Actions : passing

(4m55s sur Windows)

5.1 Tableau de bord du projet

Critere	Valeur	Statut
Tests unitaires	43 tests	Tous passes
Fichiers de documentation .Rd	18 fichiers	Complets
devtools::check()	0 errors 0 warnings 0 notes	Propre
GitHub Actions CI/CD	passing en 4m55s	Fonctionnel
Solveurs implementes	2 (ILP + C++)	Operationnels
Niveaux de jeu	3 (3x3, 4x4, 5x5)	Tous fonctionnels
Unicite des puzzles	Garantie mathematiquement	Verifiee
Fonctions exportees	14 fonctions publiques	Documentees
Lignes de code R	~1500 lignes	Commente
Lignes de code C++	~280 lignes	Commente

6. Guide d'Utilisation

6.1 Installation

```
# Installer les dependances
install.packages(c("shiny", "ggplot2", "Rcpp", "lpSolve", "DBI", "RSQLite", "testthat")
)

# Charger le package en developpement
devtools::load_all()

# Lancer l'application Shiny
run_slitherlink()
```


6.2 API Principale

```
# Creer une grille 4x4
m <- matrix(c(2,1,1,2, 1,0,0,1, 1,0,0,1, 2,1,1,2), nrow=4, byrow=TRUE)
g <- new_slitherlink(m)

# Afficher la grille
plot_slitherlink(g)

# Tracer un segment
g <- toggle_h_edge(g, 1, 1)

# Valider la solution
validate_solution(g) # retourne list(valid, messages)

# Resoudre automatiquement (ILP)
g_solved <- solve_slitherlink(g)

# Generer un puzzle avec unicite garantie
h <- matrix(0L,5,4); v <- matrix(0L,4,5)
h[1,] <- 1L; h[5,] <- 1L; v[,1] <- 1L; v[,5] <- 1L
puzzle <- generate_puzzle(h, v, difficulty = "moyen")
```

7. Dependances du Package

Package	Role	Utilisation
ggplot2	Visualisation	plot_slitherlink() — rendu de la grille
dplyr / magrittr	Manipulation donnees	Pipeline %>% dans grid_plot.R
shiny	Application web	Interface utilisateur interactive
Rcpp	Integration C++	Solveur backtracking haute performance
lpSolve	Solveur ILP	Programmation lineaire entiere
DBI / RSQLite	Base de donnees	Leaderboard persistant
testthat	Tests unitaires	Suite de 43 tests automatises

8. Workflow Git et Collaboration

8.1 Organisation des branches

Structure Git du projet

main <-- branche principale (code stable)

|

+-- dev : Phase 1 (Moussa - structure initiale)

|

+-- dev2 : Phase 2, 3, 4, 5 (tout le developpement)

Flux : dev2 -> merge -> main -> GitHub Actions -> badge vert

8.2 Bonnes pratiques appliquees

- Commits atomiques avec messages descriptifs
- Branches separees par phase / fonctionnalite
- Pull Requests pour merger sur main
- CI/CD avec GitHub Actions (R CMD check automatique)
- Badge de statut dynamique sur le README

9. Conclusion

Ce projet a permis de developper un package R complet et professionnel couvrant l'ensemble du cycle de developpement logiciel moderne :

- **du jeu Slitherlink en theorie des graphes**Modelisation mathematique rigoureuse
- **avec documentation roxygen2 complete (18 fichiers .Rd)**Architecture S3 propre
- **: backtracking C++ (Rcpp) et ILP (lpSolve)**Deux solveurs complementaires
- **mathematiquement via double resolution ILP**Unicité des puzzles garantie
- **avec design premium, timer et leaderboard SQLite**Application Shiny professionnelle
- **: 43 tests unitaires, devtools::check() propre, CI/CD GitHub Actions**Qualite garantie

Les points techniques les plus significatifs de ce projet sont l'implementation du double solveur (C++ pour la vitesse et ILP pour la sophistication mathematique), la garantie d'unicite des puzzles generes, et l'integration complete d'un pipeline de qualite logicielle (tests automatises, documentation, CI/CD) rarement vu dans un projet universitaire.

Dossou AGOSSOU & Moussa DIAGNE

Master 1 Statistique et Science des Donnees

Université de Montpellier | 17 Avril 2026